Jaimin Judd D. Roces
11/03/26
AN43

TA 4 : Implementing Knowledge Representation, Rule-Based Reasoning (RBR), and Case-Based Reasoning (CBR)

Implementation and Reflection

This activity models a laptop that powers on but does not charge. The Python program stores the problem facts in a categorized knowledge base, applies IF-THEN rules to infer actions, then retrieves a similar prior case and retains the resolved problem. The same scenario makes the two reasoning methods easier to compare

Use Case and Knowledge Base

The system assists with first-step troubleshooting of laptop charging. It records five Boolean facts in three categories: power (outlet and adapter indicator), charging (charge status and cable condition), and status (whether the laptop powers on). These facts are machine-readable key-value pairs, and the flattened fact map is shared by the rule engine and case-based system.

The facts describe one illustrative situation: the outlet works, the adapter light is off, the laptop is not charging, the cable is visibly damaged, and the laptop still powers on. The recommendations are basic troubleshooting suggestions, not a diagnosis of a specific device

Running the Program

Run TA4_Roces.py with Python 3. It prints the categorized knowledge base, inferred RBR actions, and CBR result. The retained case stays in memory for the current run.

Rule-Based Reasoning

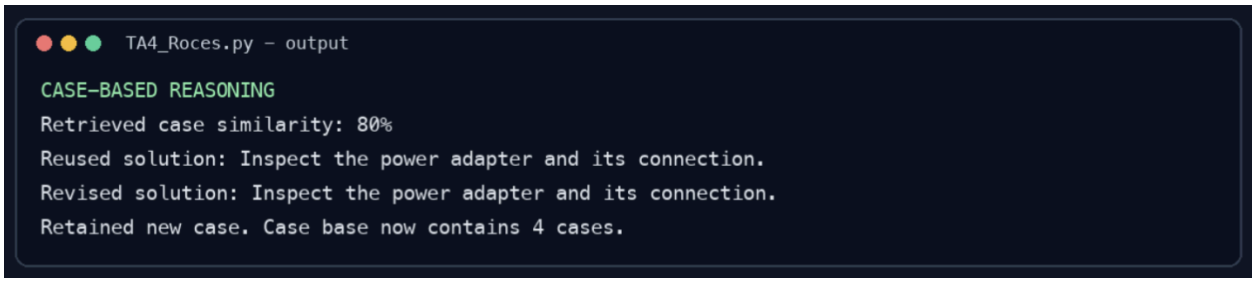
Four functions encode IF-THEN rules. The inference engine checks every rule against working memory and returns every action whose condition is true. In this example, the damaged cable, unlit adapter indicator, and charging failure each trigger a recommendation.



Console output

Case-Based Reasoning

The case base starts with three earlier laptop charging problems, each paired with a solution. Similarity is computed as the proportion of matching Boolean features. The program retrieves the highest-scoring case, reuses its solution, leaves it unchanged in the revise step, and retains the current problem as a fourth case in memory for the current run.



Retrieve, reuse, revise, and retain results

Comparative Reflection

Rule-Based Reasoning was easier to implement because each recommendation is a direct IF-THEN condition, and the inference engine checks each rule against the facts. CBR required more pieces: case records, similarity scoring, retrieval, reuse, revision, and retention.

CBR is more flexible when a new problem resembles past experience but no single rule covers it; its results still depend on relevant cases and a sound similarity measure. RBR responds directly to rapidly changing input facts when those facts are updated. CBR is useful when solutions improve through accumulated experience and resolved cases can be reused.

RBR is used in configuration checks, policy validation, and troubleshooting decision trees. CBR is used in help-desk recommendations and maintenance support based on resolved incidents.

Source Code

```
"""TA4: Knowledge Representation, Rule-Based Reasoning, and Case-Based Reasoning.

Use case: diagnosing a laptop that is not charging. The example facts and cases
are illustrative; update the student details and scenario as appropriate.
"""
from copy import deepcopy

# Knowledge representation: facts are grouped into three meaningful categories.
knowledge_base = {
    "power": {"outlet_working": True, "adapter_light_on": False},
    "charging": {"charging": False, "cable_damaged": True},
    "status": {"laptop_powers_on": True},
}

# Flatten facts for the rules and the case similarity function.
facts = {key: value for category in knowledge_base.values() for key, value in category.items()}


def rule_damaged_cable(current_facts):
    """Recommend replacing a cable when visible damage is reported."""
    if current_facts["cable_damaged"]:
        return "Replace the damaged charging cable."
    return None


def rule_check_outlet(current_facts):
    """Check the power source before diagnosing the laptop."""
    if not current_facts["outlet_working"]:
        return "Test another outlet or restore power to the outlet."
    return None


def rule_adapter(current_facts):
    """Suggest adapter inspection when its indicator stays off on a working outlet."""
    if current_facts["outlet_working"] and not current_facts["adapter_light_on"]:
        return "Inspect the power adapter and its connection."
    return None


def rule_charging_port(current_facts):
    """Check laptop-side charging when it runs but does not charge."""
    if current_facts["laptop_powers_on"] and not current_facts["charging"]:
        return "Inspect the laptop charging port and battery."
    return None


RULES = [rule_damaged_cable, rule_check_outlet, rule_adapter, rule_charging_port]


def run_inference_engine(current_facts, rules=RULES):
    """Apply every rule to working facts and return all inferred actions."""
    actions = []
    for rule in rules:
```

```
        action = rule(current_facts)
        if action is not None:
            actions.append(action)
    return actions


# Three prior cases. Each case has problem features and an associated solution.
case_base = [
    {
        "problem": {"outlet_working": True, "adapter_light_on": False,
                    "charging": False, "cable_damaged": False,
                    "laptop_powers_on": True},
        "solution": "Inspect the power adapter and its connection.",
    },
    {
        "problem": {"outlet_working": False, "adapter_light_on": False,
                    "charging": False, "cable_damaged": False,
                    "laptop_powers_on": True},
        "solution": "Test another outlet or restore power to the outlet.",
    },
    {
        "problem": {"outlet_working": True, "adapter_light_on": False,
                    "charging": False, "cable_damaged": False,
                    "laptop_powers_on": False},
        "solution": "Inspect the power adapter and its connection.",
    },
]


def similarity(problem_a, problem_b):
    """Return the fraction of matching Boolean features (range 0.0 to 1.0)."""
    features = set(problem_a) | set(problem_b)
    if not features:
        return 0.0
    matches = sum(problem_a.get(feature) == problem_b.get(feature) for feature in features)
    return matches / len(features)


def retrieve_most_similar(new_problem, cases):
    """Retrieve the highest-scoring prior case and its similarity score."""
    if not cases:
        return None, 0.0
    ranked = [(similarity(new_problem, case["problem"]), case) for case in cases]
    return max(ranked, key=lambda item: item[0])


def revise_solution(proposed_solution, revised_solution=None):
    """Reuse a proposed answer; permit a manual replacement when supplied."""
    return revised_solution.strip() if revised_solution and revised_solution.strip() else
proposed_solution


def retain_case(new_problem, final_solution, cases):
    """Add the resolved problem and solution to the case base for this run."""
    cases.append({"problem": deepcopy(new_problem), "solution": final_solution})
```

```
def run_cbr(new_problem, cases):
    """Run Retrieve, Reuse, Revise, and Retain for one new problem."""
    score, retrieved_case = retrieve_most_similar(new_problem, cases)  # Retrieve
    if retrieved_case is None:
        reused_solution = "No prior solution found; perform a new diagnosis."
    else:
        reused_solution = retrieved_case["solution"]  # Reuse
    final_solution = revise_solution(reused_solution)  # Revise (unchanged by default)
    retain_case(new_problem, final_solution, cases)  # Retain
    return score, retrieved_case, final_solution


def main():
    print("KNOWLEDGE BASE")
    for category, category_facts in knowledge_base.items():
        print(f"{category.title()}: {category_facts}")

    print("\nRULE-BASED REASONING ACTIONS")
    for action in run_inference_engine(facts):
        print(f"- {action}")

    print("\nCASE-BASED REASONING")
    score, case, solution = run_cbr(facts, case_base)
    if case:
        print(f"Retrieved case similarity: {score:.0%}")
        print(f"Reused solution: {case['solution']}")
    else:
        print("Retrieved case: none")
    print(f"Revised solution: {solution}")
    print(f"Retained new case. Case base now contains {len(case_base)} cases.")


if __name__ == "__main__":
    main()
```